

SESSION 01

The AI-First *Developer Mindset.*

How to think with AI — not just prompt it.

[30 min]

[session 1 of 5]

[pre-course]

Ahmed Qaddoura

CTO · Builder · aqaddoura.com

// the divide

Two camps of developers.

Today, you pick one.

C A M P A

The Tab Tapper

- Uses AI like autocomplete.
- Copies code without reading it.
- Ships fast. Debugs slow.
- Distrusts AI on anything serious.

Stays junior forever.

C A M P B

The Intent Architect

- Primes context before asking.
- Argues with AI. Refines answers.
- Ships code they fully understand.
- Operates AI like a senior engineer.

Compounds 5× faster. Sleeps better.

```
// the shift
```

~~You are no longer a syntax writer.~~

You are an intent architect.



```
// before
```

Translate ideas → code.

Compiler executes.

```
// now
```

 Translate ideas → clear intent.

AI executes. You verify, refine, ship.

```
// tip
```

The hard work used to be the typing. Now it's the thinking.

//where are you?

AI Developer Maturity Model

*Be honest. Then move.***L1**

The Tab Tapper

Copilot autocomplete. Paste errors into ChatGPT. Distrusts AI for anything real.

~ 10% saved**L2**

The Prompter

Knows basic prompt tricks. Still hand-fixes most output.

~ 20% saved**L3**

The Architect

Gives AI context, constraints, outcomes. Lets it propose. Argues with it.

3× output**L4**

The Operator

Memory files. Prompt libraries. Eval routines. Ships solo what required a team.

5–10× output

// the trap

Three mistakes that cap your growth.

I'm not exaggerating. I've seen all three.

01

Shipping code you can't explain

2 a.m. production bug. Your senior watching in Slack. You can't debug what you never understood.

02

Confusing speed with leverage

Faster on the wrong thing is still wrong. Leverage = right thing fast. Not just fast.

03

Outsourcing thinking, not typing

AI does syntax. If it also thinks for you — you become commodity. Markets don't pay commodity.

// the system

The PURE Loop.

Run this on every AI interaction that matters.

P**Prime**

Give AI context, stack, constraints, threat model — before any task.

U**Understand**

Read every line. Mark what's unclear. Ask 'why this over X?'

R**Refine**

Ask AI to review its own code as a senior would. Then rewrite.

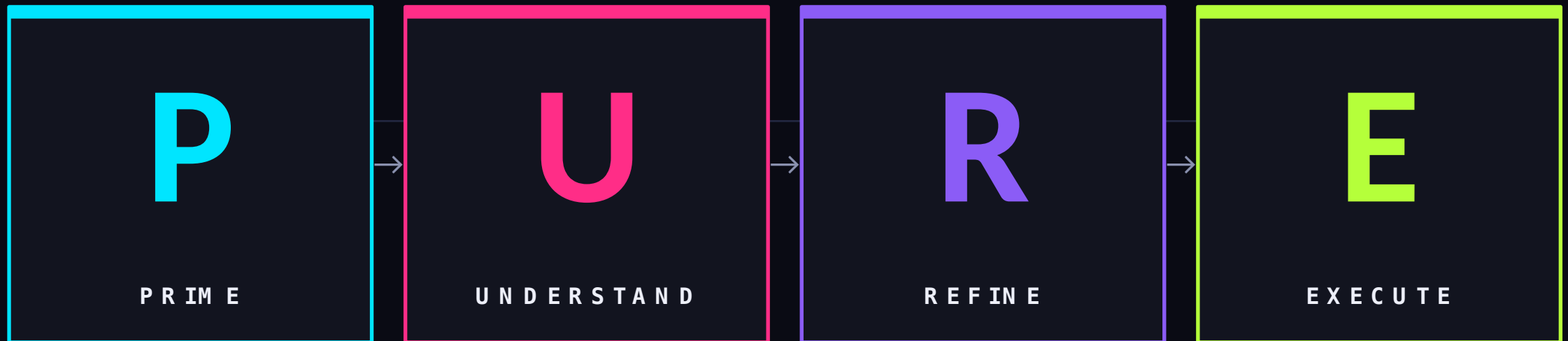
E**Execute**

Run. Measure. Write what you learned into your doctrine.

```
// the loop
```

PURE is a loop. Not a checklist.

Every Execute feeds the next Prime — the system compounds.



```
// doctrine update
```

Every Execute ends with one new rule in ai-doctrine.md. That file is your career compound interest.

// live exam ple

PURE catches 4 bugs in 3 minutes.

Task: Pydantic model for a Freelancer profile. AI's first answer looked fine.

01

Unbounded full_name

10 MB string = DoS vector. Fix: add max_length=120.

02

Email not normalized

User registers twice with different case. Fix: lower + strip.

03

Skills list unbounded

Same DoS class. Fix: cap list length (max_length=30).

04

URL scheme open

javascript: and file:// slip through. Fix: allowlist http/https.

→ Switching to screen dem o

//why now

Start compounding now.

Every month you wait is a month you don't get back.

30

D A Y S

to start seeing real leverage in your daily work

10×

C O M P O U N D R A T E

your doctrine grows sharper with every PURE loop you run

01

D O C T R I N E F I L E

your personal AI playbook — yours, nobody else's

//your first 60 minutes

Run one PURE loop on a real task.

Not a tutorial. Real work. Something you'd do this week anyway.

1

P R I M E

Write a primer: stack, style, constraints, threat model.

2

U N D E R S T A N D

Read AI's output line by line. Ask 'why this over X?' — 3 things you learned.

3

R E F I N E

Ask AI to review its own code as a senior would. Then rewrite. Compare both.

4

E X E C U T E

Use the refined output for its real purpose. Time it.

5

D O C T R I N E

Add 3 specific, reusable rules to ai-doctrine.md. This file is your asset.

//next

Session 02 — Get the best from AI tools.

Without getting burned.

01

Hidden failure modes

The 7 silent ways AI tools fail — including data leaks Camp A never notices.

02

Hallucinated APIs

Confident wrong answers. How to catch them before your build dies.

03

Local-tenant leaks

The frontend pattern AI ships in every project — and it leaks your users' data.

w a r n i n g

Skip Session 02 → you'll ship a security disaster within 90 days. I've seen 3 already this year.

Apply it.

Today — not tomorrow.

Ahmed Qaddoura

aqaddoura.com · Palestine Innovation Initiative · Taqat · Kawkab.ai

//yalla — let's build.